

2. Unterprogramme

2.1 Wie könnte die Implementierung der in der Vorlesung erwähnten Semantiken für Parameter (call-by-value und call-by-reference) in Risc-V aussehen?

In beiden Schemen können Argumente sowohl über Register als auch über den Stack übergeben werden.

Call-by-value

Der Wert wird direkt über eine vereinbarte Methode übergeben:

Per Register:

```
.data
arg0: .word 0x0ca0
arg1: .word 0x0007

.text
j main

# Expect args of integer type in a0-a7 (as per ABI)
add:
# Return value in a0 (as per ABI)
add a0, a0, a1
ret

main:
lw a0, arg0
lw a1, arg1
call add
```

Per Stack:

```
# Pass value via stack
.data
arg0: .word 0x0ca0
arg1: .word 0x0007

.text
j main

# Expect arg(n) at n(sp)
add:
# process data
# load args
lw t0 0 sp
lw t1 4 sp
add a0 t0 t1
ret

main:
lw s0, arg0
lw s1, arg1
# Allocate stack space (simplified--RISC-V ABI demands 128-bit alignment.)
addi sp, sp, -8
sw s1 4 sp
```

```
sw s0 0 sp
call add
# Deallocate stack space
addi sp sp 8
```

Call-by-reference

Die Adresse des Datums wird übergeben:

Per Register:

```
.data
arg0: .word 0x0ca0
arg1: .word 0x0007

.text
j main

add:
lw t0 0 a0 # Load value at [a0]
lw t1 0 a1
add a0, t0, t1
ret

main:
la a0, arg0
la a1, arg1
call add
```

Analog für Stack (siehe oben unter call-by-reference)

2.2 Division und Rest

a) Call-By-Value

Siehe mod.s und div.s

b) Call-By-Reference

Siehe mod_ref.s und div_ref.s

2.3 Quicksort

Siehe quicksort.s

2.4 Quicksort Stack Frame

Stackframe während der ersten Partition und des ersten Swaps:

quicksort (16 bytes)	
+12	return address -> main
+8	start address of arr (a0 arg)
+4	end address of arr (a1 arg)
partition (16 bytes)	
+12	return address -> quicksort
swap (0 bytes)	

+Offset aus der Sicht des Callees
↑ höhere Adressen ↓ niedrigere Adressen